Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación

Construcción de Compiladores

Proyecto: Intérprete SQL con Almacenamiento en Archivos

Intérprete SQL

Arquitectura, ejecución y pruebas

Compiler Lab

Estudiante
Osman Emanuel de León García
Carné 23428

Guatemala, noviembre de 2026

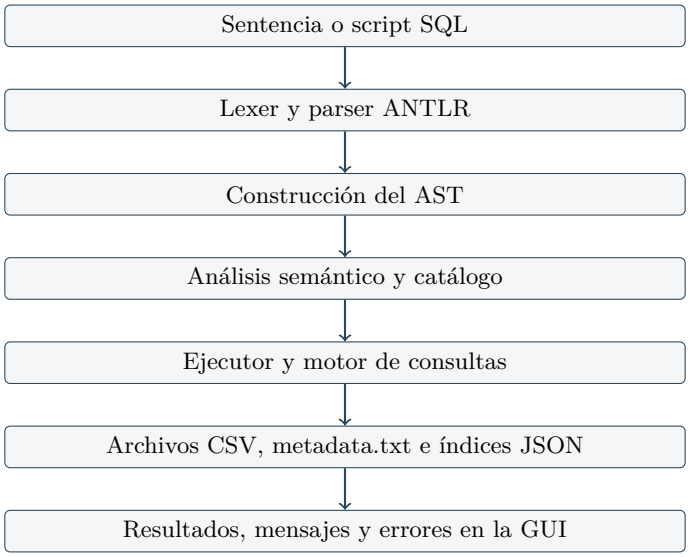
1. Introducción y alcance

Este proyecto desarrolla un intérprete de sentencias SQL que trabaja sobre un sistema de almacenamiento propio basado en archivos y directorios, sin utilizar un gestor de bases de datos externo. La implementación forma parte de *Compiler Lab*, una aplicación que también integra YALex, YAPar y Compiscript. El informe se concentra en el módulo SQL y resume su diseño, las operaciones implementadas y la validación realizada.

El sistema permite escribir SQL, visualizar tokens, árbol sintáctico y árbol de sintaxis abstracta (AST), detectar errores y ejecutar consultas desde una interfaz gráfica en Tkinter.

2. Arquitectura general

El procesamiento sigue las etapas del enunciado. La separación entre análisis y ejecución permite detectar problemas antes de modificar archivos.



Módulos principales. `frontend.py` analiza tokens y sintaxis; `ast_builder.py` construye el AST; `semantic.py` comprueba referencias, tipos y restricciones; `executor.py` coordina la ejecución; `storage.py` gestiona CSV; `query_engine.py` procesa consultas; `indexes.py` mantiene índices persistentes. La interfaz está en `gui/app.py`.

3. Implementación

3.1. Almacenamiento y tipos

Cada tabla se representa mediante un archivo CSV, mientras que `metadata.txt` conserva columnas, tipos, restricciones e índices. Los índices se almacenan como archivos JSON. Se admiten los tipos `INT`, `FLOAT`, `VARCHAR(n)`, `BOOLEAN` y `DATE`; se comprueban restricciones como `PRIMARY KEY`, `UNIQUE` y `NOT NULL`.

3.2. Operaciones SQL

Categoría	Funcionalidades
Definición	`CREATE TABLE`, `DROP TABLE`, `CREATE INDEX`, `DROP INDEX`.
Manipulación	`INSERT`, `UPDATE`, `DELETE`.
Consultas	`SELECT`, `FROM`, `WHERE`, `ORDER BY`, `INNER JOIN`.
Agregación	`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, `GROUP BY`, `HAVING`.
Subconsultas	`IN`, `NOT IN`, `EXISTS`, `NOT EXISTS` y subconsultas escalares.

Los índices relacionan valores de una columna con posiciones de registros y se reconstruyen tras inserciones, modificaciones y eliminaciones. Para condiciones simples de igualdad sobre una columna indexada, el motor consulta las posiciones candidatas del índice. Las consultas más complejas pueden requerir recorrido de filas.

3.3. Análisis semántico

Antes de ejecutar, se comprueba la existencia de tablas y columnas, la ambigüedad de referencias, la compatibilidad de tipos y la validez de los índices y las subconsultas. Las subconsultas escalares que producen más de una fila se rechazan durante la ejecución. Los errores incorporan ubicación cuando está disponible.

4. Interfaz gráfica

La aplicación ofrece un editor SQL con resaltado y números de línea, selección de carpeta de base de datos, análisis sin ejecución y ejecución explícita. Las pestañas inferiores permiten inspeccionar tokens, árbol sintáctico, AST, esquema, errores y resultados. Los resultados de `SELECT` se presentan en una tabla `ttk.Treeview`, con encabezados, desplazamiento y selector de consultas.

La barra lateral muestra únicamente los controles de la herramienta activa: SQL, Compiscript, YALex, YAPar o Entrada. Así se reduce la cantidad de acciones visibles sin alterar los motores de análisis.

5. Instalación y ejecución

Desde la carpeta principal del repositorio se instalan las dependencias del archivo `requirements.txt`. El proyecto utiliza Python y ANTLR para el análisis, y Tkinter para la interfaz.

```
python -m pip install -r requirements.txt
python ejecutar_sql.py examples/sql/demos/consultas_avanzadas.sql --db database_demo
python auditar_sql.py
```

La interfaz se inicia mediante el punto de entrada gráfico configurado en el repositorio. Para ejecutar SQL desde la ventana, se selecciona la pestaña SQL, se elige la carpeta `DATABASE`, se escribe la consulta y se pulsa `EXECUTE SQL`. La acción `ANALYZE SQL` permite revisar el código sin modificar los archivos.

6. Pruebas y resultados

Se ejecutó una batería de diez pruebas automatizadas sobre operaciones básicas, persistencia, actualización y eliminación, errores semánticos, índices, subconsultas y valores nulos. La ejecución reportada finalizó sin fallos:

```
Ran 10 tests in 0.104s
OK
```

Comprobación	**Resultado**
Crear, insertar y consultar; persistencia; UPDATE/DELETE	Correcto
Clave duplicada, tabla inexistente y tipo incompatible	Correcto
Actualización de índices y subconsulta correlacionada	Correcto
Subconsulta escalar con múltiples filas	Correcto
Distinción entre cadena vacía y `NULL`	Correcto

También se comprobó visualmente la ejecución desde Tkinter y la representación tabular de los resultados. Estas pruebas respaldan los escenarios cubiertos, pero no sustituyen una evaluación exhaustiva de todas las combinaciones de SQL.

7. Limitaciones y conclusiones

El motor se centra en el subconjunto SQL solicitado. La optimización mediante índices está implementada para búsquedas sencillas de igualdad; otros filtros pueden recorrer los registros. Además, un script con varias instrucciones no dispone de transacciones completas: si ocurre un error de ejecución, cambios anteriores pueden permanecer en disco. Los CSV creados con versiones antiguas no siempre permiten recuperar la distinción original entre `NULL` y texto vacío.

El proyecto cumple su propósito principal: analizar, validar y ejecutar instrucciones SQL mediante un mecanismo de archivos propio. La separación de lexer, parser, AST, análisis semántico, ejecución y persistencia facilita las pruebas y la extensión del sistema. La interfaz integrada permite demostrar el recorrido completo de una sentencia, desde su entrada hasta la visualización del resultado.